A Geometric Multigrid Preconditioner for Shifted Boundary Method

Michał Wichrowski[1], Ajay Ajith [2]

Abstract

Keywords: immersed boundary methods, finite element methods, Shifted Boundary Method, multigrid, patch smoothers

1. Introduction

Capturing the intrinsic detail and complexity of the real world within the rigid structure of computational grids is a persistent tension in scientific computing. Traditional finite element methods (FEM) act like artisanal tailors, requiring body-fitted meshes that must conform precisely to every contour of the domain's boundary. While effective, this bespoke mesh generation is computationally expensive and frequently becomes the bottleneck for intricate three-dimensional shapes. Unfitted finite element methods offer a liberating alternative by employing a fixed background mesh that remains blissfully agnostic to the physical boundary. Among these, the Shifted Boundary Method (SBM) [29] adopts a distinct strategy: instead of the mesh chasing the boundary, the boundary is mathematically shifted to meet the mesh. By defining the problem on a surrogate domain and extrapolating boundary conditions, SBM avoids the complex geometric intersections of methods like CutFEM [15]. However, in a display of the "conservation of difficulty," SBM essentially shifts part of the challenge from the mesh generator to the linear solver. The resulting linear systems are burdened with extrapolation terms that introduce non-symmetry and potential indefiniteness, proving that a simple mesh does not necessarily guarantee a simple matrix. While conditioning issues scale like $\mathcal{O}(h^{-2})$ [6], similar to body-fitted methods [19], the efficient numerical solution of algebraic systems emanating from high-order SBM formulations remains somewhat unexplored.

Despite its advantages, the development of robust and scalable solvers for SBM, particularly geometric multigrid preconditioners [24], has remained an open problem. In previous work [34], a geometric multigrid preconditioner was developed for a Discontinuous Galerkin (DG) SBM formulation. That approach leveraged the inherent block structure of DG to utilize element-wise smoothers. However, while effective for low orders, that strategy faced significant difficulties at higher polynomial degrees ($p = 3$), where the simple cell-wise smoother failed to resolve the error efficiently, leading to deteriorating iteration counts. For Continuous Galerkin (CG) methods, the strong coupling between degrees of freedom makes element-wise smoothing even less viable. Consequently, this work aims to not only address the specific challenges of classical SBM but also to overcome the high-order robustness issues encountered in previous formulations by introducing a more sophisticated patch-based smoothing strategy.

In SBM, the *surrogate domain* $\tilde{\Omega}$ is typically constructed as a union of cells from a fixed background mesh that are deemed *active* (e.g., entirely inside or significantly intersecting the true domain Ω), and its boundary $\tilde{\Gamma}$ does not conform to the domain boundary Γ. Boundary conditions are transferred from the true to the surrogate boundary, typically via Taylor expansions or more general extension operators [45], and enforced in a Nitsche-like manner [31]. The Shifted Boundary Method has evolved from its first formulation [29, 30], which used cells strictly within the considered domain, to a recent approach [44] that often includes intersected cells based on a volume fraction threshold. It has been extended to high-order discretizations [7], various physical problems including Stokes flow [3], solid mechanics [6, 8], and significantly, to problems with embedded interfaces [28, 41]. These latter works demonstrate the capability of SBM to handle discontinuities across internal boundaries by appropriately modifying the formulation to impose jump conditions, extending

[1]Heidelberg University, Germany, mt.wichrowsk@uw.edu.pl
[2]Heidelberg University, Germany

the method's applicability to multiphysics and multi-material problems. Other innovations include penalty-free variants [19] and integration with level set methods [27, 43].

While SBM avoids the complexities of generating body-fitted meshes, the primary geometric task shifts to accurately determining the relationship between points on the surrogate boundary $\tilde{\Gamma}$ and the true boundary Γ. The method inherently allows for the use of arbitrarily complex geometries, and crucially, avoids the need to compute integrals over the arbitrarily shaped integration domains that arise from cell-boundary intersections. While determining the active mesh requires computing volume fractions of cut cells, this is a one-time geometric preprocessing step. In contrast, methods like CutFEM require specialized quadrature for all terms in the bilinear form on every cut cell. This makes SBM significantly more efficient in terms of computational throughput, especially in matrix-free implementations where the overhead of cut-cell quadrature would be incurred in every operator evaluation. The SBM utilizes closest-point projection algorithms to find for each point on $\tilde{\Gamma}$ a corresponding point on Γ, but other strategies could be used, such as level sets. Level set methods, which represent the domain boundary as the zero level set of a function, are often employed in unfitted methods like SBM to facilitate operations such as closest point projection [27, 43]. Special treatment of domains with corners was analyzed in [5].

MW: level set is just a very simple way to describe domain, so that statement should be corrected

MW: maybe cite literature on closest point that is common in graphics?

However, the geometric flexibility of SBM comes at the cost of new computational challenges. The extrapolation of boundary conditions and the resulting modifications to the variational formulation often lead to linear systems with non-symmetric and potentially indefinite properties [34], particularly for higher-order polynomial approximations. These properties make the efficient solution of the resulting system non-trivial and demand specialized preconditioning strategies.

Multigrid methods [13, 24] are renowned for their potential to solve large systems arising from partial differential equations, often offering optimal or near-optimal complexity. However, their application as preconditioners for SBM remains largely unexplored. While Algebraic Multigrid (AMG) has been applied to SBM for discretizations using continuous linear elements [4], its efficiency for higher-order methods is not established. In fact, AMG generally struggles with high-order finite element discretizations even for body-fitted meshes, and for unfitted methods like SBM, it is even less effective for $p > 1$. Furthermore, AMG may not fully leverage the geometric information in structured background meshes. Geometric multigrid methods, in contrast, explicitly use the hierarchy of meshes and can be very effective. A critical component is the smoother. Standard smoothers like Jacobi or Gauss-Seidel, however, struggle with the non-standard local properties of SBM systems. They implicitly enforce zero correction on the boundary of the local support (e.g., a vertex patch for Gauss-Seidel), which conflicts with the non-zero boundary terms introduced by SBM.

To address these issues in the context of Continuous Galerkin SBM, we adopt the perspective of subspace correction methods [42, 12]. The effectiveness of a smoother depends critically on the choice of subspaces and the local problems solved on them. Since simple one-dimensional subspaces (corresponding to individual DoFs) are ineffective for SBM due to the boundary conflicts mentioned above, we construct local problems on larger, overlapping subspaces defined by patches of elements centered around vertices. This concept is akin to overlapping Schwarz smoothers [32], but tailored for the specific challenges of SBM. By solving a local problem on a patch that includes the relevant SBM boundary terms, we can compute a more effective correction. We introduce a *shyness* criterion to ensure that patches are only formed around vertices sufficiently surrounded by active cells, thereby preventing the creation of ill-conditioned local problems on small, isolated slivers of the domain.

The adoption of patch-based smoothers is further justified by recent algorithmic improvements that render them highly efficient, particularly in matrix-free settings. While traditional overlapping Schwarz methods are often viewed as computationally expensive, recent work on smoothers with localized residual computations [38] demonstrates that multiplicative formulations allow for the fusion of residual computation with the local subspace correction. This significantly reduces memory transfers, a key bottleneck on modern hardware. Furthermore, the local problems on these patches can be solved with remarkable efficiency; interior patches can leverage fast tensor-product inverses [40, 20], while more general patches can be solved using local p-multigrid techniques. This approach yields p-robustness at a cost comparable to a global operator evaluation [37, 35]. Indeed, recent results for CutFEM on GPUs [20] validate the effectiveness of such vertex-patch smoothers for unfitted methods, reporting promising iteration counts and demonstrating that performing multiple smoothing sweeps specifically on boundary patches is a viable strategy to ensure robustness. Thus, while this work utilizes matrix-based implementations to demonstrate the robustness of the multigrid approach, the proposed smoother is designed to align with future high-performance, matrix-free

solvers that avoid expensive matrix assembly [26, 39, 40].

In the broader context of unfitted methods, CutFEM [15] is a prominent alternative with theoretical results for preconditioners already developed. It discretizes directly on the physical domain by cutting background cells, requiring specialized quadrature. Due to inherent difficulties with small cuts, proper stabilization seems to be an unavoidable part of CutFEM. The so-called ghost penalty [14, 36] solves the issue of ill-conditioning but may require additional care to avoid locking [9, 11, 17]. CutFEM has been applied to various problems, including Stokes [16], elasticity [25], or two-phase flows [18]. Furthermore, Discontinuous Galerkin methods have also been combined with CutFEM [23, 11]. While CutFEM ensures robust conditioning via geometry-adapted quadrature, SBM retains the efficiency comparable to standard tensor-product quadrature.

Concerning preconditioning, CutFEM seems to pose challenges. Results providing optimal preconditioners [22, 21] have been developed. Although these methods are shown to be mesh-independent, iteration counts can be high. In [11] DG-CutFEM was considered, and a multigrid preconditioner based on cell-wise Additive Schwarz smoother was used. Although the paper mostly focuses on matrix-free implementation, the preconditioner seems promising. However, the smoothing step requires a rather high number of matrix-vector products.

This paper addresses the computational challenges of solving linear systems arising from Continuous Galerkin SBM discretizations. Our main contribution is the development of a novel subspace correction smoother, termed the *Full-Residual Shy Patch* smoother. This smoother is built on local problems defined on overlapping patches of elements, constructed to be faithful representations of the global SBM problem. We introduce a *shyness* criterion to ensure robustness and avoid ill-conditioning on small patches. We demonstrate that this patch-based smoother, combined with a multi-stage strategy, leads to a highly effective h-multigrid preconditioner. Numerical experiments demonstrate the preconditioner's effectiveness under mesh refinement and compare our results with an algebraic multigrid (AMG) preconditioner, highlighting the limitations of AMG for higher-order SBM discretizations. The implementation is built on the `deal.II` finite element library [2, 1], leveraging its comprehensive tools for finite element methods and multigrid.

The remainder of this paper is organized as follows. In Section 2, we briefly review the Shifted Boundary Method formulation to establish the necessary notation and context. Section 3 details the core of our contribution: the construction of the Full-Residual Shy Patch smoother and the associated multigrid hierarchy. The key concept of *Shy Patches*, which ensures robustness by avoiding ill-conditioned slivers near the boundary, is formally introduced in Section 3.2. We then present numerical evidence in Section 5, demonstrating the effectiveness of the method and providing comparisons with AMG and CutFEM. In essence, this work demonstrates that while SBM matrices can be notoriously difficult to handle at high polynomial degrees, they can be effectively tamed by our *Shy Patch* smoothers. By solving local problems on vertex patches while carefully avoiding ill-conditioned regions, we achieve a solver that remains robust in both mesh size h and polynomial degree p. This robustness is clearly illustrated by the stable iteration counts presented in Figure 4 and Table 4, while the comparative performance against other methods is summarized in Table 5. Finally, Section 6 offers concluding remarks.

2. Method formulation

We consider the Poisson problem as a model problem:

$$-\Delta u = f \quad \text{in } \Omega, \tag{1}$$

$$u = g \quad \text{on } \Gamma, \tag{2}$$

where $\Omega \subset \mathbb{R}^d$ $(d = 2, 3)$ is a domain with boundary $\Gamma = \partial\Omega$ as depicted in Figure 1, f is a given source term, and g is the prescribed Dirichlet boundary condition.

To solve this problem numerically, we first formulate it in a weak sense. We seek a solution u in an appropriate function space, $H^1(\Omega)$, which consists of functions that are square-integrable and whose first derivatives are also square-integrable. Multiplying the equation by a test function $v \in H^1(\Omega)$ and integrating over Ω, we obtain:

$$\int_\Omega \nabla u \cdot \nabla v \, dx - \int_\Gamma (\nabla u \cdot \mathbf{n})\, v \, ds = \int_\Omega f \, v \, dx.$$

We next introduce a triangulation $\mathcal{T}_h$ consisting of quadrilateral (2D) or hexahedral (3D) elements of size h, and define a finite element space $\mathbb{V}_h \subset H^1(\Omega)$ using Lagrange polynomial elements of degree p. In classical finite element methods, the mesh conforms to the boundary (unlike the background mesh approach illustrated in Figure 1), and test functions v typically vanish on Γ to strongly enforce the Dirichlet condition $u = g$. When function spaces do not necessarily satisfy essential boundary conditions strongly, boundary conditions must be enforced weakly through additional integral terms on Γ.

Nitsche's method provides a way to weakly impose Dirichlet boundary conditions within a variational formulation without requiring the function space to satisfy the boundary conditions. It modifies the bilinear form by adding terms on the boundary Γ. The standard Nitsche formulation for the Poisson problem with Dirichlet boundary conditions $u = g$ on Γ seeks $u_h \in V_h$ such that for all $v_h \in \mathbb{V}_h$:

$$\int_\Omega \nabla u_h \cdot \nabla v_h \, dx \; - \int_\Gamma (\nabla u_h \cdot \mathbf{n}) v_h \, ds - \alpha \int_\Gamma (\nabla v_h \cdot \mathbf{n}) u_h \, ds + \int_\Gamma \sigma u_h v_h \, ds = \\ = \int_\Omega f v_h \, dx - \int_\Gamma (\nabla v_h \cdot \mathbf{n}) g \, ds + \int_\Gamma \sigma_\Gamma g v_h \, ds.$$

Here, σ_Γ is a penalty parameter, typically chosen as $\sigma_\Gamma = \mathcal{O}(p^2 h^{-1})$ for mesh size h, and $\mathbf{n}$ is the outward unit normal vector to Γ. The choice of parameter $\alpha = 1$ leads to a symmetric formulation, while $\alpha = -1$ results in a non-symmetric formulation in which the penalty term can be skipped [10]

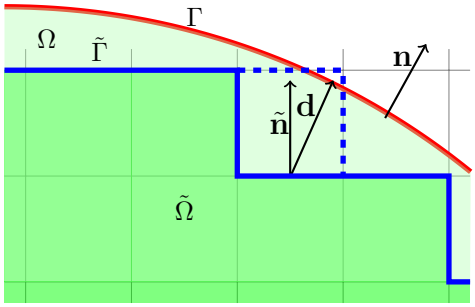

Figure 1: Schematic illustrating the background mesh, interior cells (green), the surrogate boundary $\tilde{\Gamma}$ (thick blue line) along the upper boundary of the interior cells, and the true boundary $\partial\Omega$ (red).

2.1. Shifted Boundary Method

MW: This section needs to be altered, for not it is a copy-paste from the previous paper.

In many applications, the domain Ω may have a complex geometry, making the generation of body-fitted meshes challenging. Non-body-fitted (unfitted) methods address this challenge by employing a background mesh $\mathcal{T}_h$ that does not conform to the boundary of Ω. The domain Ω is embedded within this background mesh, and cells of $\mathcal{T}_h$ are classified as active based on their intersection with Ω. The computational domain $\tilde{\Omega}$ is defined as the union of these active cells. In the original SBM formulation [29], only cells strictly contained in Ω were included, leading to the surrogate domain boundary depicted by the solid blue line in Figure 1. Consequently, the surrogate domain $\tilde{\Omega}$ is generally a subset of Ω, and its boundary $\tilde{\Gamma}$ does not coincide with the true boundary Γ.

Later extensions include intersected cells [44] with a volume fraction inside Ω greater than a threshold $\lambda \in [0, 1]$. In Figure 1, this corresponds to including additional cells as indicated by the dashed blue line. This approach reduces the distance between true and surrogate boundaries, but can lead to ill-conditioning.

To impose boundary conditions on $\tilde{\Gamma}$, the Dirichlet condition prescribed on the true boundary Γ is extrapolated to the surrogate boundary. This is typically accomplished by projecting points from $\tilde{\Gamma}$ onto the true boundary Γ along a suitable direction (e.g., the outward normal to $\tilde{\Gamma}$) and using a Taylor expansion to approximate the boundary values. This is accomplished via an *extension operator* $\mathcal{E}$, which maps functions defined on $\tilde{\Omega}$ to the surrogate boundary $\tilde{\Gamma}$.

We assume that the Dirichlet boundary condition g is given as a restriction of a function $u^\star$ defined on the entire domain Ω to the boundary Γ. While the choice of this function $u^\star$ (as an extension of g from Γ into Ω) is not unique, we take $u^\star$ to be equal to the solution u in the surrogate domain $\tilde{\Omega}$. For each point

$\tilde{\mathbf{x}} \in \tilde{\Gamma}$, let $\mathbf{x} \in \Gamma$ be its closest point projection onto the true boundary, and let $\mathbf{d} = \mathbf{x} - \tilde{\mathbf{x}}$ be the shift vector. The function u is extended from Γ to $\tilde{\Gamma}$ using a Taylor expansion:

$$\mathcal{E}u^\star(\tilde{\mathbf{x}}) = u^\star(\mathbf{x}) - \mathbf{d} \cdot \nabla u(\tilde{\mathbf{x}}) + \cdots$$

where $\mathcal{E}u^\star$ denotes the extrapolated boundary condition. The function $u^\star$ is assumed to be smooth in a neighborhood of $\tilde{\Gamma}$, which allows for the Taylor expansion to be valid. By substituting it into the weak formulation, we obtain a variational formulation for the shifted boundary problem with the extrapolated boundary condition on $\tilde{\Gamma}$ enforced in a Nitsche-like manner; the weak formulation seeks $u_h \in V_h$ such that for all $v_h \in V_h$,

$$\begin{aligned}\int_{\tilde{\Omega}} \nabla u_h \cdot \nabla v_h \, dx - \int_{\tilde{\Gamma}} (\nabla u_h \cdot \tilde{\mathbf{n}}) v_h \, ds - \alpha \int_{\tilde{\Gamma}} (\nabla v_h \cdot \tilde{\mathbf{n}}) \, \mathcal{E}u_h \, ds + \int_{\tilde{\Gamma}} \sigma \, \mathcal{E}u_h \, v_h \, ds = \\ = \int_{\tilde{\Omega}} f v_h \, dx - \int_{\tilde{\Gamma}} (\nabla v_h \cdot \tilde{\mathbf{n}}) g^E \, ds + \int_{\tilde{\Gamma}} \sigma g v_h \, ds.\end{aligned} \tag{3}$$

where $\tilde{\mathbf{n}}$ is the outward normal to $\tilde{\Gamma}$ and σ is a penalty parameter.

The choice of the stabilization term, particularly the parameter α, significantly influences the spectral properties of the resulting system matrix. As detailed in our previous work on a Discontinuous Galerkin (DG) based SBM multigrid preconditioner [34], this can be observed even in a simple 1D single-cell problem. For instance, a penalty-free formulation ($\sigma = 0$, $\alpha = -1$) can lead to complex eigenvalues, especially when the true boundary lies inside the surrogate domain.

In the context of DG methods, it was found that the choice of stabilization did not have a dramatic impact on the overall multigrid performance, as the cell-based nature of the DG smoother effectively handled local issues [34]. However, in the DG formulation the cellwise smoother experienced singifficant challenges for $p = 3$. For continuous finite elements, where our smoother operate on larger, overlapping patches of elements (i.e. vertex or edge patches), the choice of stabilization becomes far more critical. The non-standard terms introduced by SBM are not well-contained within these patches, and an inappropriate stabilization can introduce spectral properties that standard smoothers cannot handle effectively, a problem that is exacerbated for higher-order elements.

The SBM weak formulation with symmetrized stabilization seeks $u_h \in V_h$ such that for all $v_h \in V_h$,

$$\begin{aligned}\int_{\tilde{\Omega}} \nabla u_h \cdot \nabla v_h \, dx - \int_{\tilde{\Gamma}} (\nabla u_h \cdot \tilde{\mathbf{n}}) v_h \, ds - \int_{\tilde{\Gamma}} (\nabla v_h \cdot \tilde{\mathbf{n}}) \, \mathcal{E}u_h \, ds + \int_{\tilde{\Gamma}} \sigma_\Gamma \, \mathcal{E}u_h \, \mathcal{E}v_h \, ds = \\ = \int_{\tilde{\Omega}} f v_h \, dx - \int_{\tilde{\Gamma}} (\nabla v_h \cdot \tilde{\mathbf{n}}) g^E \, ds + \int_{\tilde{\Gamma}} \sigma_\Gamma g v_h \, ds.\end{aligned} \tag{4}$$

where $\tilde{\mathbf{n}}$ is the outward normal to $\tilde{\Gamma}$ and σ_Γ is a penalty parameter. We notice that our discrete solution u_h is a piecewise polynomial function defined on the background mesh $\mathcal{T}_h$, hence its Taylor expansion can be computed directly by evaluating the function values at the points on the true boundary Γ. This allows us to avoid computation of higher-order derivatives of u_h. Note that the resulting form is not symmetric due to the presence of the extrapolated boundary condition.

3. Multigrid Preconditioner

The SBM formulation described above leads to a large, sparse linear system. Due to the lack of symmetry and possible indefiniteness of the resulting matrix, we employ a Krylov subspace method, specifically GMRES, for its solution. The convergence of GMRES is sensitive to the condition number of the system matrix, which grows with both the number of elements in the mesh and the polynomial degree. To accelerate the solution, we employ a multigrid preconditioner [24]. Our Cartesian background mesh naturally facilitates the construction of a nested hierarchy of meshes $\{\mathcal{T}_\ell\}_{\ell=0}^{L}$, where ℓ denotes the level and L is the finest level. On each mesh $\mathcal{T}_\ell$, we define a finite element space $\mathbb{V}_\ell$ consisting of continuous piecewise polynomials of a fixed degree p.

This represents an important difference from the work on a Discontinuous Galerkin SBM preconditioner [34], where an hp-multigrid strategy with lower polynomial degrees on coarser levels was necessary to

achieve good performance. In the present continuous Galerkin context, we find that a simpler h-multigrid approach is enough. We note that while the background meshes are nested, the sets of active cells on different levels (determined by the fraction λ of their volume inside the domain) are not necessarily nested. This lack of geometric nestedness can affect the efficiency of standard inter-grid transfer operators.

With this hierarchy of meshes and spaces established, the multigrid method is defined through three crucial components: a smoother, which reduces high-frequency error components on each level; transfer operators, which move information between different resolution levels; and a coarse-grid solver. For preconditioning, we use a single multigrid V-cycle.

3.1. Smoother

We first consider Richardson iterations with preconditioners P. Given the current approximation u_h and the right-hand side f_h, a smoothing step updates the approximation:

$$u_h^{k+1} = u_h^k + P(f_h - A_h u_h^k).$$

We decompose the space $\mathbb{V}_\ell$ into a sum of subspaces $\mathbb{V}_i$, i.e., $\mathbb{V}_\ell = \sum_{i=1}^N \mathbb{V}_i$. Let A_i be the restriction of A to the subspace $\mathbb{V}_i$. Then, the additive subspace correction preconditioner is defined as:

$$B = \omega \sum_{i=1}^{N} A_i^{-1}$$

where ω is a relaxation parameter. Alternatively, the successive subspace correction method is a subspace correction method where the subspaces $\mathbb{V}_i$ are visited in a sequential manner. The procedure can be performed by applying one Richardson iteration with the preconditioner P_i for each subspace $\mathbb{V}_i$. Since in each step only one subspace is corrected, the residual only changes locally and an efficient implementation is possible. The preconditioner updates the solution by sequentially applying corrections for each subspace. For each subspace $\mathbb{V}_i$, a correction is computed and applied:

$$u_h \leftarrow u_h + R_i^T A_i^{-1} R_i (f_h - A_h u_h).$$

This process is repeated for all subspaces $i = 1, \ldots, N$. If the subspaces are chosen as one-dimensional spaces spanned by the basis functions, then the preconditioner is equivalent to either Jacobi (additive) or Gauss-Seidel (successive).

Viewing the smoother as a subspace correction method provides a useful framework. The effectiveness of such a smoother depends critically on the choice of subspaces and the local problems solved on them. For continuous finite element methods, simple choices like one-dimensional subspaces corresponding to individual degrees of freedom (leading to Jacobi or Gauss-Seidel smoothers) are ineffective for SBM. These methods implicitly enforce a zero correction on the boundary of the single-vertex *patch*, which conflicts with the non-standard boundary terms introduced by SBM, a fact confirmed by our preliminary experiments.

This suggests that the subspaces must be large enough to capture the local behavior of the SBM formulation. We therefore construct local problems on larger subspaces, defined by patches of elements centered around a vertex, as illustrated in Figure 2. By solving a local problem on this patch that includes the relevant SBM boundary terms where applicable, we can compute a more effective correction. This approach aims to make the local problem on the patch a more consistent approximation of the global problem, leading to a more robust smoother.

3.2. Full-Residual Shy Patches

The idea behind our proposed smoother is to construct local problems on patches of cells that are consistent with the global problem, particularly in handling the non-standard terms introduced by the SBM. We start by constructing vertex patches, where a patch consists of all active cells connected to a given vertex. The subspace for the smoother correction is then defined by the degrees of freedom (DoFs) supported by the cells within this patch.

Next we ensure that the local problem solved on the patch is a faithful representation of the global problem. To achieve this, we only include DoFs for which the complete residual can be evaluated using

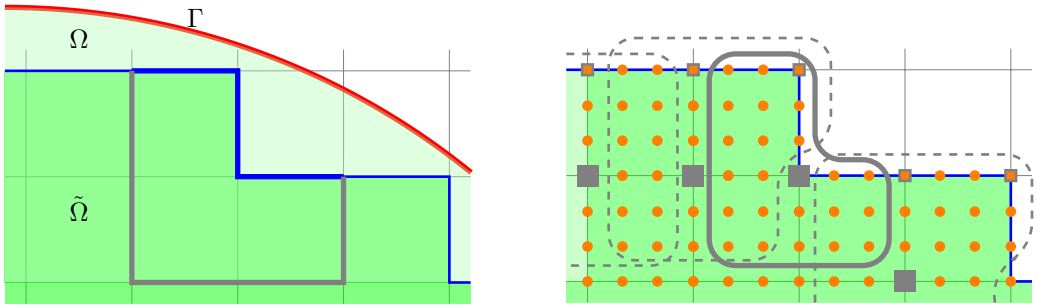

Figure 2: Left: Boundaries of a vertex patch. Interior boundaries shown in light gray, and the thick blue segments indicate the exterior faces where SBM boundary conditions are applied. Right: the same vertex patch with a gray frame highlighting which degrees of freedom form the patch subspace; three other patches are indicated with gray dashed outlines, and the central vertices are marked by filled gray squares. The vertices marked with small gray squares may be too shy to form their own patches, depending on the shyness threshold. Note that as long as the shyness threshold is at most 3 all degrees of freedom from those patches will be include in other patches.

information solely from the cells within the patch. This means we exclude any DoFs whose support extends to cells outside the current patch. By doing so, we guarantee that the local correction is computed using the true global residual, avoiding inconsistencies that would arise from incomplete information, especially near the surrogate boundary. Figure 2 illustrates this concept for a few patches.

We further refine this patch selection with a concept we call *shyness*. Much like a shy person at a party needs a circle of friends to feel confident enough to join the dance, our vertices are considered shy. A vertex will only form the center of a patch if it is surrounded by a sufficient number of *friends* — in this case, active cells. We define a shyness threshold as the minimum number of active cells required around a vertex to consider it for patch construction. If a vertex does not meet this threshold, it is deemed too isolated, and no patch is formed around it. This strategy prevents the creation of small local problems, which reduces the computational cost of the smoother and avoids potential issues with ill-conditioning that can arise from patches with too few active cells.

This raises an important question about the maximum shyness threshold required to ensure that every degree of freedom is included in at least one patch. If a DoF is not part of any patch, its value will never be updated by the smoother, which is detrimental to convergence. In two dimensions, a vertex can be adjacent to at most four cells. As illustrated in Figure 2, if a vertex is adjacent to only one or two active cells, the DoFs associated with that vertex might not be included in any other patch if their respective central vertices are also *shy*. To guarantee that every DoF belongs to at least one patch, the shyness threshold ξ must be at most 3 in 2D, while in 3D it must be at most 4. This ensures that even if a vertex is too shy to form its own patch, its associated DoFs are guaranteed to be included in the patch of a sufficiently *sociable* neighboring vertex.

In [34], satisfactory results were obtained with 3 smoothing steps. A multi-stage smoothing strategy was proposed in [20] for CutFEM, where an initial global smoothing pass was followed by additional passes restricted to patches containing cut cells. We adopt a similar strategy here. The first smoothing step is applied to all patches in the domain. Subsequent steps are then selectively applied only to those patches that contain cells adjacent to the surrogate boundary. This focuses the computational effort of the smoother on the region where the SBM introduces non-standard terms and where the error is often most difficult to resolve. These boundary-adjacent patches are identified once during the construction of the smoother.

3.3. Transfer Operators

The transfer of information between different levels of the multigrid hierarchy is handled by prolongation and restriction operators. The prolongation operator, $I_{\ell-1}^{\ell}$, maps a function from the coarse space $\mathbb{V}_{\ell-1}$ to the fine space $\mathbb{V}_{\ell}$, while the restriction operator, $I_{\ell}^{\ell-1}$, transfers a function from the fine space to the coarse space. For standard multigrid methods on nested meshes with continuous finite elements, prolongation is typically the natural embedding of the coarse function space into the fine one, and restriction is its transpose.

A significant challenge in applying multigrid to unfitted methods like SBM is the lack of geometric nestedness of the computational domains. The set of active cells $\tilde{\Omega}_{\ell}$ on a given level ℓ is determined independently

based on the intersection with the true domain Ω. Consequently, the active domain on the fine level, $\tilde{\Omega}_L$, is not necessarily a subset of the active domain on a coarser level, $\tilde{\Omega}_\ell$ (when viewed on the fine grid). This can lead to inconsistencies where, for example, an active fine-grid cell corresponds to a non-active coarse-grid cell.

This lack of nestedness can impair the effectiveness of standard transfer operators [34]. Information from an active region on the fine grid might be restricted to a non-active region on the coarse grid, where it is essentially discarded, breaking the flow of information required for an efficient multigrid cycle.

4. Implementation details

The numerical implementation of our multigrid solver is based on the open-source finite element library `deal.II` [1]. It provides a comprehensive framework for the implementation of finite element methods, including mesh handling, finite element spaces, assembly of linear systems, and interfaces to various linear algebra solvers and preconditioners. In this paper we build upon the implementation of the multigrid solver for DG-SBM presented in [34].

4.1. Background mesh and geometry handling

When implementing SBM on a non-body-fitted mesh, we need to handle cells intersected by the true boundary Γ. We use a level set function $\phi(\mathbf{x})$ to implicitly define the domain $\Omega = \{\mathbf{x} \mid \phi(\mathbf{x}) < 0\}$, with $\Gamma = \{\mathbf{x} \mid \phi(\mathbf{x}) = 0\}$. Cells of the background mesh are classified based on their intersection with the zero level set: cells entirely inside Ω (interior), cells entirely outside Ω (exterior), and cells intersected by Γ. Then, for the intersected cells the fraction of the cell volume inside Ω is computed, and the cell is classified as active if this fraction is greater than a threshold λ. This is handled using non-matching quadrature rules implemented in `deal.II`, which are based on the techniques described in [33].

In our approach, degrees of freedom are formally assigned to all cells of the background mesh, including those that are classified as non-active. In the matrix assembly we ignore the contributions from these non-active cells, effectively removing them from the system. This results in a singular global matrix; however, since the corresponding degrees of freedom do not influence the solution in the active domain, this does not pose a problem for the iterative solver. The only affected part of the multigrid is the coarse-grid solver, which we handle by using a direct solver. To make the coarse system invertible, we set one on the diagonal entries of zero rows corresponding to non-active DoFs.

4.2. Processing surrogate boundary and matrix assembly

The SBM requires computing the closest point projection from points on the surrogate boundary $\tilde{\Gamma}$ to the true boundary Γ. This projection is found by solving a local nonlinear optimization problem for each quadrature point on $\tilde{\Gamma}$, which minimizes the distance to the true boundary defined by the zero level set of a function $\phi(\mathbf{x})$. The full details of the formulation, which involves a Lagrange multiplier approach solved with a Newton-Raphson method, are described in [34]. To ensure sufficient smoothness for the derivative calculations required by the solver, the level set function $\phi(\mathbf{x})$ is represented using finite elements of degree 2 for $p = 1$ and degree p for $p > 1$. The search for the closest point is performed only in the interior of the cells adjacent to the surrogate boundary. While this does not guarantee that the closest point is found inside the cell, it is expected that even if the closest point is outside the cell, the extrapolation will still yield a good approximation of the boundary condition.

The extrapolation of the function values from the true boundary to the surrogate boundary required for the matrix assembly process is accomplished by evaluating the values of the basis functions at the points on the surrogate boundary. The assembly of the cell contributions and interior faces to the system matrix and right-hand side vector is performed using the standard finite element assembly process provided by `deal.II`.

4.3. Multigrid structures

For the multigrid hierarchy, we leverage `deal.II`'s built-in capabilities for handling nested meshes and defining finite element spaces on each level. The standard projection operators provided by `deal.II` are used for the prolongation ($I^\ell_{\ell-1}$) and restriction ($I^{\ell-1}_\ell$) operators, transferring data between coarser and finer grid levels. These operators act on the entire background mesh, transferring the solution for all degrees of freedom,

irrespective of whether they correspond to active or non-active cells. This involves transferring values in regions outside the computational domain. Furthermore, since the smoother and residual evaluations are restricted to the active cells, the values in the inactive regions do not propagate into the solution within the domain of interest, nor do they influence the convergence of the method.

The assembly of the system matrix A_ℓ on each level ℓ of the multigrid hierarchy follows a procedure analogous to that on the finest level. The level set function defining the domain geometry is interpolated onto the mesh $\mathcal{T}_\ell$. Based on this interpolated level set and the chosen threshold λ, active cells for level ℓ are identified. The SBM bilinear form is then used to assemble the local contributions to A_ℓ only for these active cells. The cells deemed non-active on level ℓ are ignored during the assembly, effectively decoupling their degrees of freedom from the system on that level.

5. Numerical results

In this section, we present numerical results to evaluate the performance of the proposed SBM multigrid preconditioner for the Poisson equation. We investigate its effectiveness in terms of convergence rates, iteration counts under mesh refinement (h-refinement) and polynomial degree increase (p-refinement). To better illustrate the multigrid preconditioner's performance characteristics and enable meaningful comparison of iteration counts, we solve the system to high accuracy with a tolerance of 10^{-12} for the relative residual reduction. This stringent tolerance results in higher iteration counts, making the differences in solver performance more apparent and facilitating clearer assessment of the multigrid effectiveness. Solver failure is defined as exceeding 100 GMRES iterations without reaching this tolerance. The initial background mesh is a square (or cube in 3D) domain covering the range $[-1.01, 1.01]$ in each dimension, subdivided into 4 cells in each coordinate direction. We use a penalty parameter $\sigma = 5$ in the SBM formulation. This parameter is selected to be sufficiently large to ensure stability of the Nitsche coupling on the surrogate boundary.

The majority of our tests are conducted on a unit sphere (a unit disk in 2D, depicted on the left panel of Fig. 3). While geometrically simple, the unit sphere serves as an insightful benchmark. Any sufficiently smooth (C^1) complex boundary, when viewed at a fine enough mesh resolution, locally resembles a flat plane. The unit sphere, due to its uniform curvature, presents a comprehensive range of intersection angles between the true boundary and the background mesh cells. Furthermore, the boundary intersects cells at various locations relative to cell centers and faces, leading to a diverse distribution of shift vector magnitudes and directions. This includes scenarios where the shift vector points from the surrogate boundary towards the interior of the true domain (which we denote as negative shifts if they oppose the outward normal of the surrogate boundary). The right panel of Figure 3 depicts the minimum and maximum shift magnitudes observed across the surrogate boundary for a typical discretization. The shift magnitudes are normalized by the cell size h; for a unit cell, the theoretically largest possible shift magnitude in 2D is $\sqrt{2}h$, occurring if the surrogate boundary point is at a cell corner and the true boundary passes through the diagonally opposite corner.

The primary metric for evaluating the multigrid preconditioner is the number of GMRES iterations required to reduce the initial residual by a factor of 10^{-8}. If the threshold is not met within 100 iterations, the solver is considered to have failed. All experiments are conducted using the implementation described in the previous section. As our solver depends on the value of the threshold parameter λ, we will first explore the solver performance on 2D problems as they are less computationally intensive and then show some results for 3D problems with tuned λ.

MW: don't touc it for now. We might want lambda here

MW: TODO: describe the RH

MW: we have two metrics: estimate of max eigenvalues of Smoother applied to A and number of GMRES iterations

5.1. Iteration counts

Figure 4 illustrates the iteration counts with respect to the number of refinement levels for both 2D and 3D problems. The results demonstrate the h-robustness of the proposed preconditioner, as the iteration counts remain largely independent of the mesh size for a fixed polynomial degree. We observe a mild increase in iterations as the polynomial degree p increases, which is expected when using h-multigrid for high-order discretizations without p-coarsening. However, patch-smoother for matching meshes has been shown to be robust even in p [38, 32], suggesting that further improvements may be possible with additional refinements to the smoother or the multigrid strategy.

We further investigate the influence of the number of smoothing steps s on the solver performance. Table 1 presents the iteration counts for varying s with a fixed shyness threshold $\xi = 3$. In the 2D case

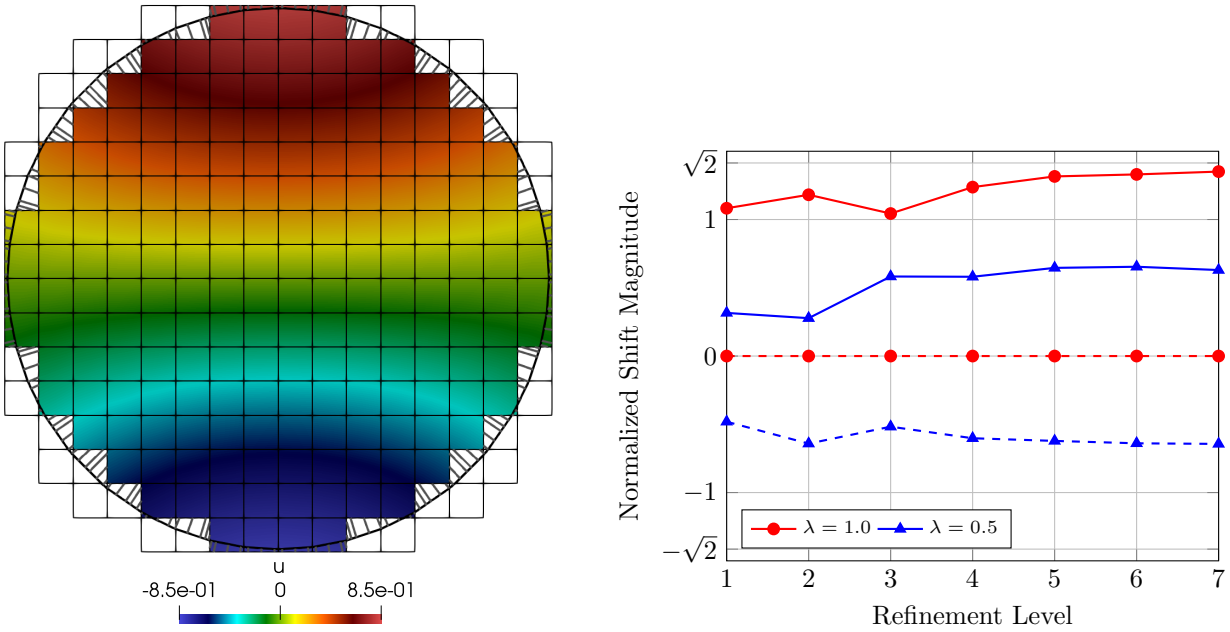

Figure 3: Left: Exemplary numerical solution for the Poisson equation on a unit disc domain, visualized on the surrogate domain ($\lambda = 0.75$), obtained with the shifted boundary method. The true boundary Γ is shown in black lines; connections between quadrature points ($p = 2$) on the surrogate boundary and their projections on the true boundary are illustrated with grey lines. Note that with $\lambda = 0.75$, some parts of the true boundary lie within the surrogate domain Right: Illustrative distribution of minimum and maximum shift magnitudes (normalized by cell size h) on the surrogate boundary for the unit disk problem with $p = 3$. The values $\pm\sqrt{2}$ represent the theoretical maximum possible normalized shift magnitude in 2D for a square cell.

(Table 1a), for $p = 1$, the solver is robust even with a single smoothing step. However, for $p = 2$ and $p = 3$, increasing the number of smoothing steps is crucial. For $p = 3$, the solver fails with $s = 1$ but converges rapidly with $s = 2$ or $s = 3$. A similar trend is observed in the 3D case (Table 1b), where increasing s consistently reduces the iteration count, particularly for higher polynomial degrees.

Table 1: Iteration counts for varying smoothing steps s with fixed shyness threshold $\xi = 3$. Left: 2D problem ($L = 8$). Right: 3D problem ($L = 4$).

(a) 2D problem ($L = 8$)

$p \setminus s$	1	2	3
1	11	11	11
2	22	13	11
3	—	29	16

(b) 3D problem ($L = 4$)

$p \setminus s$	1	2	3
1	13	11	10
2	15	10	9
3	28	15	11

Finally, we analyze the effect of the shyness threshold ξ on the convergence, with the number of smoothing steps fixed at $s = 3$. The results are summarized in Table 2. For the 2D problem (Table 2a), we present results for the baseline threshold $\xi = 3$. In the 3D case (Table 2b), we compare thresholds $\xi = 3, 4, 5$. The results indicate that while the method is robust for $\xi = 3$ and $\xi = 4$, increasing the threshold to $\xi = 5$ results in solver failure. This suggests that a threshold of $\xi = 5$ is too restrictive, leaving some degrees of freedom outside of any patch, thereby degrading the quality of the smoother.

We also examine the influence of the cell threshold parameter λ on the solver performance. Table 3 presents the iteration counts for varying λ with fixed shyness threshold $\xi = 3$ and smoothing steps $s = 3$. In the 2D case (Table 3a), for $p = 1$ and $p = 2$, the solver is relatively robust to λ, although $\lambda = 1.0$ yields slightly lower iteration counts. However, for $p = 3$, small values of λ ($\lambda \leq 0.5$) lead to increased iterations or failure, while $\lambda \geq 0.75$ restores convergence. In the 3D case (Table 3b), the sensitivity is more pronounced. For $p \geq 2$, $\lambda = 0.5$ results in solver failure, while $\lambda = 1.0$ provides robust convergence.

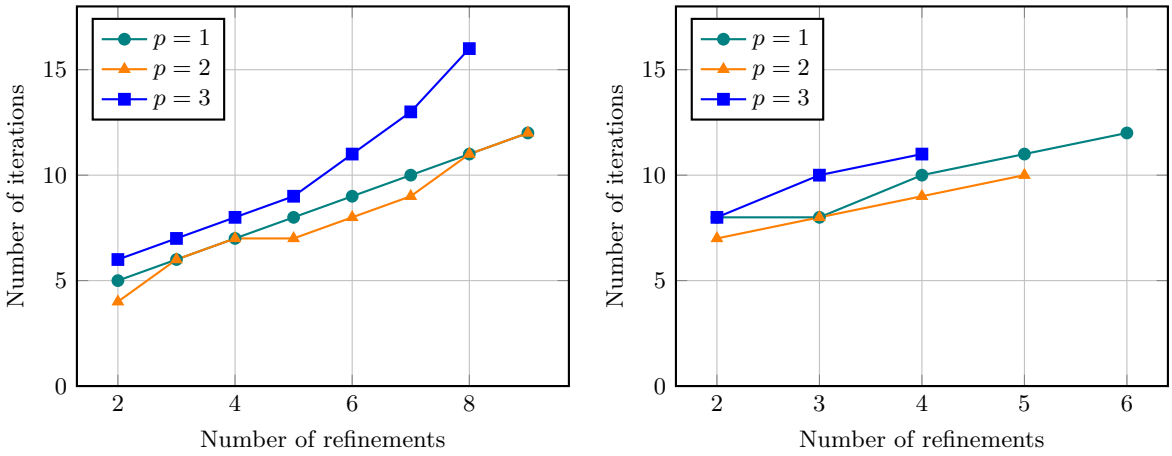

Figure 4: Iteration counts for multigrid preconditioned GMRES solver using the Full-Residual Shy Patch smoother with varying shyness thresholds equal to 3 on unit ball problem for polynomial degrees $p = 1, 2, 3$. Number of smoothing steps is fixed to 3. Left panel presents results in 2D with shyness 3, right panel in 3D with shyness 4.

Table 2: Iteration counts for varying shyness threshold ξ with fixed smoothing steps $s = 3$. Left: 2D problem ($L = 8$). Right: 3D problem ($L = 4$).

(a) 2D problem ($L = 8$)

$p \setminus \xi$	1	2	3	4
1	11	11	11	—
2	10	10	10	—
3	15	15	15	—

(b) 3D problem ($L = 4$)

$p \setminus \xi$	3	4	5
1	10	10	—
2	9	9	—
3	11	11	—

Table 3: Iteration counts for varying cell threshold λ with fixed shyness threshold $\xi = 3$ and smoothing steps $s = 3$. Left: 2D problem ($L = 8$). Right: 3D problem ($L = 4$).

(a) 2D problem ($L = 8$)

$p \setminus \lambda$	0.25	0.5	0.75	1.0
1	13	13	13	11
2	12	11	11	11
3	—	27	14	16

(b) 3D problem ($L = 4$)

$p \setminus \lambda$	0.5	1.0
1	13	10
2	—	9
3	—	11

5.2. p-Multigrid

We observed a mild increase in iteration counts with increasing polynomial degree for the h-multigrid method. This is likely due to the properties of the transfer operators between the grid levels. To obtain a method that is robust with respect to the polynomial degree and does not rely on a hierarchy of nested meshes, we implemented a p-multigrid strategy. In this approach, the grid is fixed, and the multigrid hierarchy is constructed by coarsening the polynomial degree of the finite element space. The coarsest level corresponds to $p = 1$, which is solved using a direct solver. Currently, this strategy is implemented only for 2D problems, as the direct solver for the $p = 1$ system on the fine grid becomes prohibitively expensive in 3D. However, this limitation can be overcome by using an algebraic multigrid (AMG) solver for the coarse $p = 1$ problem, which would extend the applicability of the p-multigrid method to 3D.

Table 4 presents the iteration counts for the p-multigrid solver for polynomial degrees $p = 2, 3, 4$ and varying mesh refinement levels. The results demonstrate excellent robustness with respect to both the mesh size h and the polynomial degree p. The iteration counts are very low and remain nearly constant as the mesh is refined, confirming the effectiveness of the p-multigrid approach combined with the Shy Patch smoother.

Table 4: Iteration counts for p-multigrid preconditioned GMRES solver with varying polynomial degree p and mesh refinements. Shyness threshold $\xi = 3$, smoothing steps $s = 3$, cell threshold $\lambda = 1.0$.

$p \setminus L$	2	3	4	5	6	7	8
2	4	4	4	4	4	3	3
3	4	4	4	4	4	3	3
4	4	4	5	5	5	5	5

5.3. Comparison with other methods

Table 5: GMRES iteration counts (tolerance 10^{-8}) for the Poisson problem. Comparison of the continuous SBM ($\lambda = 1$) preconditioned with the Full-Residual Shy Patch smoother (shyness $\xi = 3$, $s = 3$, 2D) against DG-SBM with a cellwise SSOR smoother ($s = 3$) and algebraic multigrid (AMG). Solver failures (no convergence within 100 iterations) are indicated by "—". Empty entries indicate that the data is not available.

Formulation	Preconditioner	Degree	Mesh refinement						
			2	3	4	5	6	7	8
SBM, $\lambda = 1$	h-MG,Shy-Patch smoother	1	5	5	7	8	9	10	11
		2	4	6	7	7	8	9	10
		3	5	7	8	9	11	13	15
	p-MG, Shy-Patch smoother	2	4	4	4	4	4	3	3
		3	4	4	4	4	4	3	3
		4	4	4	5	5	5	5	5
	AMG	1	2	2	12	13	14	15	17
DG-SBM, $\lambda = 0.75$	cellwise-SSOR,	1	5	6	7	8	9	9	11
	hp-multigrid	2	7	9	10	12	14	14	16
cutFEM	h-MG, Patch smoother[20]	1				6	6	6	5
		2				9	8	7	7
		3				17	14	13	13

To place the performance of the proposed SBM preconditioners in context, we compare them against other established methods for the 2D Poisson problem. Table 5 summarizes the iteration counts for the continuous SBM formulation alongside results for Discontinuous Galerkin SBM (DG-SBM), Algebraic Multigrid (AMG), and CutFEM. We focus on the 2D case to enable a comparison across a broad range of refinement levels.

We first consider the DG-SBM formulation solved using an *hp*-multigrid method with a cellwise SSOR smoother. The proposed SBM with the Shy-Patch smoother consistently requires fewer iterations than DG-SBM. This advantage becomes increasingly apparent as the mesh is refined, underscoring the superior smoothing capabilities of the patch-based approach compared to cellwise smoothers within the SBM framework.

Next, we benchmark against a standard AMG preconditioner for linear finite elements ($p = 1$). While AMG is highly efficient on coarser meshes, its iteration counts tend to grow steadily with mesh refinement. In contrast, the geometric multigrid with the Shy-Patch smoother exhibits a much more moderate increase in iterations, suggesting better scalability properties on fine meshes.

Finally, we compare our approach with the Cut Finite Element Method (cutFEM) utilizing a patch smoother [20]. Our method yields iteration counts comparable to cutFEM. While cutFEM shows remarkable stability for $p = 1$, our h-multigrid approach experiences a slight increase in iterations. However, the p-multigrid variant (p-MG) demonstrates exceptional robustness, outperforming all other tested methods with very low and nearly constant iteration counts across all refinement levels and polynomial degrees.

5.4. Timing

Finally, we evaluate the computational efficiency of the proposed solver. Figure 5 presents a comparison of the throughput (Degrees of Freedom per second) between our multigrid preconditioner with the Full-Residual Shy Patch smoother and an algebraic multigrid (AMG) preconditioner. Although AMG currently shows higher throughput for $p = 1$, it is worth noting that patch-based smoothers are particularly advantageous for matrix-free implementations. Recent studies [38, 20, 35, 37] have demonstrated that matrix-free geometric multigrid methods with patch smoothers can achieve high performance on modern architectures by minimizing memory traffic. Our current implementation is matrix-based, and we expect that a matrix-free optimization would yield significant performance improvements.

We further analyze the cost of performing additional smoothing steps. Figure 6 displays the relative increase in computational time when increasing the number of smoothing steps from one to two. The plotted value corresponds to the ratio $(T_{s=2}/T_{s=1}) - 1$, representing the cost of the second (and consecutive) sweeps relative to the first one. As detailed in Section 3.2, the Shy Patch smoother restricts subsequent relaxation sweeps to only those patches affected by the boundary. Consequently, the cost of these additional sweeps is substantially lower than that of the initial full sweep. Furthermore, as the mesh is refined, the fraction of patches near the boundary diminishes, leading to a decreasing relative cost for the extra smoothing steps. Given the significant reduction in iteration counts achieved with multiple smoothing steps (see Table 1), this marginal additional cost is well justified, particularly for higher polynomial degrees.

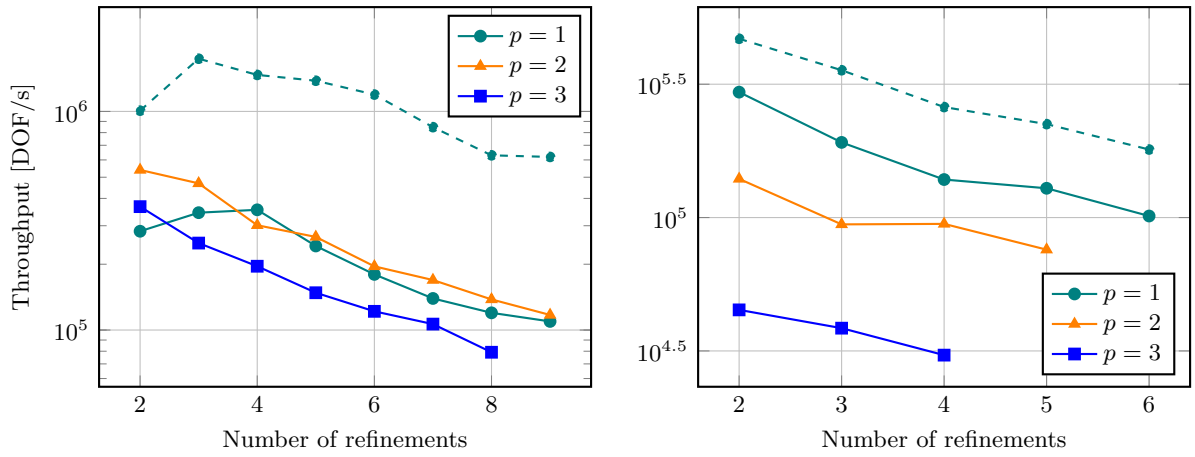

Figure 5: Comparison of throughput (DOF/s) for the proposed multigrid preconditioner with Full-Residual Shy Patch smoother ($\xi = 3$, $s = 3$) against an algebraic multigrid (AMG) preconditioner from `deal.II`'s interface to `Trilinos`. The tests are performed on the unit ball problem in 2D for polynomial degrees $p = 1, 2, 3$ with $s = 3$ smoothing steps.

6. Conclusion

Acknowledgments

The author declares the use of language models (ChatGPT, Gemini, and Claude) to improve the clarity and readability of the manuscript. All scientific content and technical claims are solely the responsibility of the author.

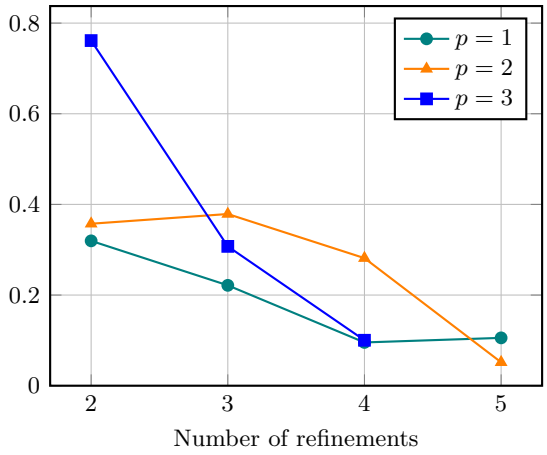

Figure 6: Ratio of the time per iteration for 2 smoothing steps vs 1 smoothing step ($xi = 4$ threshold 1.0).

References

[1] D. Arndt, W. Bangerth, M. Bergbauer, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, M. Kronbichler, M. Maier, et al., *Thedeal. ii library, version 9.7*, Journal of Numerical Mathematics, (2025).

[2] D. Arndt, W. Bangerth, D. Davydov, T. Heister, L. Heltai, M. Kronbichler, M. Maier, J.-P. Pelteret, B. Turcksin, and D. Wells, *The deal.II finite element library: Design, features, and insights*, Computers & Mathematics with Applications, 81 (2021), pp. 407–422.

[3] N. Atallah, C. Canuto, and G. Scovazzi, *Analysis of the shifted boundary method for the Stokes problem*, Computer Methods in Applied Mechanics and Engineering, 358 (2020), p. 112609.

[4] N. Atallah, C. Canuto, and G. Scovazzi, *The second-generation shifted boundary method and its numerical analysis*, Computer Methods in Applied Mechanics and Engineering, 372 (2020), p. 113341.

[5] N. Atallah, C. Canuto, and G. Scovazzi, *Analysis of the shifted boundary method for the Poisson problem in domains with corners*, Mathematics of Computation, 90 (2021), pp. 2041–2069.

[6] N. Atallah, C. Canuto, and G. Scovazzi, *The shifted boundary method for solid mechanics*, International Journal for Numerical Methods in Engineering, 122 (2021), pp. 5935–5970.

[7] N. Atallah, C. Canuto, and G. Scovazzi, *The high-order shifted boundary method and its analysis*, Computer Methods in Applied Mechanics and Engineering, 394 (2022), p. 114885.

[8] N. Atallah and G. Scovazzi, *Nonlinear elasticity with the shifted boundary method*, Computer Methods in Applied Mechanics and Engineering, 426 (2024), p. 116988.

[9] S. Badia, E. Neiva, and F. Verdugo, *Linking ghost penalty and aggregated unfitted methods*, Computer Methods in Applied Mechanics and Engineering, 388 (2022), p. 114232.

[10] C. E. Baumann and J. T. Oden, *A discontinuous hp finite element method for convection—diffusion problems*, Computer Methods in Applied Mechanics and Engineering, 175 (1999), pp. 311–341.

[11] M. Bergbauer, P. Munch, W. A. Wall, and M. Kronbichler, *High-performance matrix-free unfitted finite element operator evaluation*, arXiv preprint arXiv:2404.07911, (2024).

[12] J. H. Bramble, J. E. Pasciak, and J. Xu, *The analysis of multigrid algorithms with nonnested spaces or noninherited quadratic forms*, Mathematics of Computation, 56 (1991), pp. 1–34.

[13] A. Brandt, *Multi-level adaptive solutions to boundary-value problems*, Mathematics of computation, 31 (1977), pp. 333–390.

[14] E. Burman, *Ghost penalty*, Comptes Rendus. Mathématique, 348 (2010), pp. 1217–1220.

[15] E. Burman, S. Claus, P. Hansbo, M. G. Larson, and A. Massing, *CutFEM: discretizing geometry and partial differential equations*, International Journal for Numerical Methods in Engineering, 104 (2015), pp. 472–501.

[16] E. Burman and P. Hansbo, *Fictitious domain methods using cut elements: III. A stabilized Nitsche method for Stokes' problem*, ESAIM: Mathematical Modelling and Numerical Analysis, 48 (2014), pp. 859–874.

[17] E. Burman, P. Hansbo, and M. G. Larson, *On the design of locking free ghost penalty stabilization and the relation to CutFEM with discrete extension*, arXiv preprint arXiv:2205.01340, (2022).

[18] S. Claus and P. Kerfriden, *A CutFEM method for two-phase flow problems*, Computer Methods in Applied Mechanics and Engineering, 348 (2019), pp. 185–206.

[19] J. H. Collins, A. Lozinski, and G. Scovazzi, *A penalty-free shifted boundary method of arbitrary order*, Computer Methods in Applied Mechanics and Engineering, 417 (2023), p. 116301.

[20] C. Cui and G. Kanschat, *A multigrid method for cutfem and its implementation on gpu*, arXiv preprint arXiv:2508.11608, (2025).

[21] S. Gross and A. Reusken, *Optimal preconditioners for a Nitsche stabilized fictitious domain finite element method*, arXiv preprint arXiv:2107.01182, (2021).

[22] S. Gross and A. Reusken, *Analysis of optimal preconditioners for CutFEM*, Numerical Linear Algebra with Applications, 30 (2023), p. e2486.

[23] C. Gürkan and A. Massing, *A stabilized cut discontinuous Galerkin framework for elliptic boundary value and interface problems*, Computer Methods in Applied Mechanics and Engineering, 348 (2019), pp. 466–499.

[24] W. Hackbusch and W. Hackbusch, *The Multi-Grid Method of the Second Kind*, Multi-Grid Methods and Applications, (1985), pp. 305–353.

[25] P. Hansbo, M. G. Larson, and K. Larsson, *Cut finite element methods for linear elasticity problems*, in Geometrically Unfitted Finite Element Methods and Applications: Proceedings of the UCL Workshop 2016, Springer, 2017, pp. 25–63.

[26] M. Kronbichler and K. Kormann, *Fast matrix-free evaluation of discontinuous Galerkin finite element operators*, ACM Transactions on Mathematical Software (TOMS), 45 (2019), pp. 1–40.

[27] D. Kuzmin and J.-P. Bäcker, *An unfitted finite element method using level set functions for extrapolation into deformable diffuse interfaces*, Journal of Computational Physics, 461 (2022), p. 111218.

[28] K. Li, N. Atallah, A. Main, and G. Scovazzi, *The shifted interface method: a flexible approach to embedded interface computations*, International Journal for Numerical Methods in Engineering, 121 (2020), pp. 492–518.

[29] A. Main and G. Scovazzi, *The shifted boundary method for embedded domain computations. Part I: Poisson and Stokes problems*, Journal of Computational Physics, 372 (2018), pp. 972–995.

[30] A. Main and G. Scovazzi, *The shifted boundary method for embedded domain computations. Part II: Linear advection–diffusion and incompressible Navier–Stokes equations*, Journal of Computational Physics, 372 (2018), pp. 996–1026.

[31] J. A. Nitsche, *Über ein Variationsprinzip zur Lösung von Dirichlet-Problemen bei Verwendung von Teilräumen, die keinen Randbedingungen unterworfen sind*, Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg, 36 (1971), pp. 9–15.

[32] L. F. Pavarino, *Additive schwarz methods for the p-version finite element method*, Numerische Mathematik, 66 (1993), pp. 493–515.

[33] R. I. Saye, *High-order quadrature methods for implicitly defined surfaces and volumes in hyperrectangles*, SIAM Journal on Scientific Computing, 37 (2015), pp. A993–A1019.

[34] M. Wichrowski, *A geometric multigrid preconditioner for discontinuous galerkin shifted boundary method*, arXiv preprint arXiv:2506.12899, (2025).

[35] M. Wichrowski, *Local solvers for high-order patch smoothers via p-multigrid*, arXiv preprint arXiv:2510.17785, (2025).

[36] ——, *Matrix-Free Ghost Penalty Evaluation via Tensor Product Factorization*, arXiv preprint arXiv:2503.00246, (2025).

[37] ——, *Multigrid p-robustness at jacobi speeds: Efficient matrix-free implementation of local p-multigrid solvers*, arXiv preprint arXiv:2512.02577, (2025).

[38] M. Wichrowski, P. Munch, M. Kronbichler, and G. Kanschat, *Smoothers with localized residual computations for geometric multigrid methods for higher-order finite elements*, SIAM Journal on Scientific Computing, 47 (2025), pp. B645–B664.

[39] M. Wichrowski, M. Rezaee-Hajidehi, J. Korelc, M. Kronbichler, and S. Stupkiewicz, *Matrix-free methods for finite-strain elasticity: Automatic code generation with no performance overhead*, International Journal for Numerical Methods in Engineering, 126 (2025), p. e70166.

[40] J. Witte, D. Arndt, and G. Kanschat, *Fast tensor product Schwarz smoothers for high-order discontinuous Galerkin methods*, Computational Methods in Applied Mathematics, 21 (2021), pp. 709–728.

[41] D. Xu, O. Colomés, A. Main, K. Li, N. Atallah, N. Abboud, and G. Scovazzi, *A weighted shifted boundary method for immersed moving boundary simulations of Stokes' flow*, Journal of Computational Physics, 510 (2024), p. 113095.

[42] J. Xu, *The method of subspace corrections*, Journal of Computational and Applied Mathematics, 128 (2001), pp. 335–362.

[43] T. Xue, W. Sun, S. Adriaenssens, Y. Wei, and C. Liu, *A new finite element level set reinitialization method based on the shifted boundary method*, Journal of Computational Physics, 438 (2021), p. 110360.

[44] C.-H. Yang, K. Saurabh, G. Scovazzi, C. Canuto, A. Krishnamurthy, and B. Ganapathysubramanian, *Optimal surrogate boundary selection and scalability studies for the shifted boundary method on octree meshes*, Computer Methods in Applied Mechanics and Engineering, 419 (2024), p. 116686.

[45] R. Zorrilla, R. Rossi, G. Scovazzi, C. Canuto, and A. Rodríguez-Ferran, *A shifted boundary method based on extension operators*, Computer Methods in Applied Mechanics and Engineering, 421 (2024), p. 116782.